dgf-factory: AI agents on rails for DGF configuration

A concept paper for heads of delivery, architects and technology leads

dotGov Solutions — dgf-factory
3 November 2026

Executive summary
1. The problem
2. The idea
3. What changes for a delivery team
4. Why it is credible
5. What it deliberately does not do
6. Risks, costs and containment
7. Decisions asked of the reader
Sources

Executive summary

Most of a DotGov Framework (DGF) application is configuration, not code: forms, workflows, processes, entity settings and component JSON (ADR 0010). AI coding agents are increasingly asked to write that configuration, and nothing checks it the way the DGF runtime will read it. The schemas DGF ships accept JSON the runtime never reads (ADR 0011). They cannot see a behaviour that changed between releases (ADR 0013). They cannot tell whether a process transition names a state that exists (ADR 0014). For workflows, the grammar rejects most of DGF's own samples (ADR 0016). An agent's change can pass every check available today and still fail on the first request, or be silently ignored.

dgf-factory gives the agent a fixed route and checks every step of it. It takes the plan → implement → verify → commit pipeline of AI Factory, an open-source agent toolkit. It ships that pipeline with a DGF knowledge base in which every fact carries the DGF version it was read from. Scripts, not the model, decide every question a script can decide. They cover both schema families, check whether the runtime actually reads a format, resolve references the way the engine does, and block only on what the branch introduced. A learning loop turns each fix into a project rule. A rule may only tighten the next run, never loosen it.

It is built and tested. Twenty-two skills, the validators and the knowledge base install into a project for any of 16 AI-tool targets, Claude Code and Codex among them (ADR 0029). Its claims are limited on purpose. It checks configuration statically and never runs it. In its change pipeline it writes no C#, SQL or Angular. It cannot check roles, because nothing in a DGF workspaces root declares them (ADR 0014 §5, ADR 0010 §3, ADR 0027 §6).

The ask. Approve a time-boxed pilot on one brownfield estate. Before it starts, refresh the knowledge base to that estate's DGF version. Name an owner for the estate's rules and for the knowledge refresh. Decide how the package is distributed. Section 7 sets these out.

1. The problem

DGF's shipped checks miss six kinds of mistake that an agent writing DGF configuration can make. Each comes from the decision records behind this tool, and each was measured against DGF's source or its sample estate.

JSON that validates but the runtime ignores. DGF publishes a JSON schema for 66 components, `Workflow` and `ProcessFlow` among them (knowledge base: component catalogue §1). For those two the runtime reads only `_workflow.xml` and `process.xml`. A JSON workflow can pass its schema and never be loaded (ADR 0011; knowledge base: schema families §6). Of the 67 rows in DGF's own format-coverage table, 57 are marked supported, 7 partial and 2 not supported; one is struck through. Only that table says which is which: whether a schema exists says nothing about it.

A reader that differs from the schemas. The runtime's JSON reader skips an unknown property without a word, accepts property names in any case, and does not read a number written as a string (knowledge base: JSON reader §1). DGF's own schema validator strips the schemas' ban on unknown properties, so a misspelt property passes it and then has no effect; 42 sample DataTables carry a `showHeaders` property the runtime ignores. Read literally, the schemas reject every one of the 230 sample files whose type could be resolved (ADR 0031).

Behaviour that changes between releases. In DGF 1.1.14, `paging` on an eager TreeTable was "accepted in configuration and then ignored". In 1.1.15 the same JSON is a hard configuration error. Both validate against the same schema, so schema validation cannot stand in for knowing the release (ADR 0013; blueprint open question 6).

Process references the grammar cannot check. The process grammar has no key or key-reference constraints. It cannot tell that a transition names a declared state, and DGF's cross-reference validator leaves processes out. A workflow that moves a case to a state the process no longer declares passes every shipped check. At run time the engine throws `New state '…' information not found`. A process also cannot run headlessly: stepping one needs a case record, so it needs the database (ADR 0014).

Grammars that lag the runtime. Measured over DGF's samples, `workflow.xsd` rejects 211 of 341 workflow files, and the grid-form and table-view grammars reject 54% and 43% of theirs. The engine runs those files. A team that switches on strict XSD validation gets noise, and soon switches it off (ADR 0016).

A shared base and an old estate. A file in the `webasm` base workspace is live in every application that inherits it, so a change there can break an application the author never opened (ADR 0026; knowledge base: composition specs §1.1). Real estates already carry findings. On a copy of DGF's samples the validators report more than 2,500 that predate any change (ADR 0018; see Measurements). A check that blocks on all of them fails every branch, and a check that fails every branch gets ignored.

A general-purpose agent brings none of this knowledge. Asked how many components DGF has, it has five different correct answers to choose from: 34, 67, 56, 59 or 66. Each counts something different (knowledge base: component catalogue §1). Its output reaches review looking plausible, and the reviewer must catch what no tool did.

2. The idea

dgf-factory joins a proven pipeline to DGF-specific knowledge and checks. Five ideas carry it.

2.1 A pipeline that keeps its state on disk

AI Factory routes every change through the same steps, and each step writes a file the next one reads. A session can be cleared, can crash or can be resumed days later. Each file has exactly one writer, so two steps never overwrite each other's work. Every quality gate ends in a machine-readable result block, not prose. dgf-factory keeps all of this (blueprint, Part 1) and gives the steps DGF meaning:

Step	What it does for a DGF change
`/dgf`	Inventories the workspaces root and records which DGF release the estate runs (ADR 0019)
`/dgf-plan`	Writes a plan that names every workspace it touches, says why if `webasm` is one, and marks each task as configuration or code, with a reason for code (ADR 0009, ADR 0017)
`/dgf-implement`	Composes each task: modern JSON where the runtime reads JSON, legacy XML where it does not, a little JavaScript or CSS only where configuration cannot express the change (ADR 0010)
`/dgf-verify`	The gate: every task done, every changed file inside the plan, and no new validator finding across the whole root (ADR 0018, ADR 0022)
`/dgf-commit`	Conventional commits, grouped by workspace as the plan's commit plan says

Around this spine sit `/dgf-fix` and `/dgf-evolve` (the learning loop, §2.5). Four skills each work on one kind of DGF artefact: `/dgf-component`, `/dgf-process`, `/dgf-model` and `/dgf-audit` (ADR 0027). Nine quality skills review, test-plan, document and research, and none of them is a gate (ADR 0032, ADR 0034). `/dgf-scaffold` creates a new application in an empty folder (ADR 0026, ADR 0027 §7).

2.2 DGF knowledge that ships, stamped with its version

AI Factory learns a project at setup. dgf-factory arrives already knowing DGF: the schema families and parity table, the component catalogue, workspace layout and naming, how the runtime reads JSON, the process and data models, every reference a file can make, permissions and application layout, with the vendored schemas (69 JSON schemas, 9 XSDs). Every fact was read from DGF's source or documentation and carries the DGF version and date it was read. A maintainer-only ledger records each source file with a digest, so an upstream change can be detected without re-reading prose (ADR 0013; knowledge base: README §1). Only facts whose behaviour changed between releases carry a version range: ranging everything "to be safe" would make the whole base go stale (ADR 0013 §2).

2.3 Determinism before prompting

The largest departure from AI Factory: if a schema, a grammar or a script can decide a question, a script decides it. The plan's format and scope, the route to JSON or XML, every finding, the gate's status and the suggested next command all come from scripts; the verify skill only relays a block a script computed. A block never claims a check that did not run: a missing baseline or dependency is a named warning, never a pass (ADR 0022). The model keeps the work that needs judgement: turning a request into a plan, choosing components, writing the configuration, explaining.

2.4 Both schema families, runtime parity, and the engine's own resolution rules

The validators check what the runtime will do, not only what a schema allows:

- **Family first, never JSON by default.** Every file's family, JSON or legacy XML, is resolved before it is parsed. A tool that assumes JSON makes every legacy file look broken (knowledge base: schema families §2).
- **A schema pass is never reported as runtime support.** A configuration valid in a family the runtime does not read is a named warning. The route for a new file comes from the parity table, and the five legacy artefact types stay XML, as DGF's current policy says (ADR 0011; knowledge base: composition specs §4).
- **References resolve the way the engine resolves them.** That means one path per form of reference, exact case and no fallback. A dead transition or a workflow that moves a case to an undeclared state blocks. A reference that resolves in some applications but not others is a warning that names them (ADR 0014). The data model gets the same treatment: a missing reference blocks only where the runtime's loader would throw (ADR 0027 §3).
- **Grammars that lag are advisory.** A failure against a lagging grammar is a warning with the grammar's own message. Only the process grammar, which has a list of the places where it differs from the runtime, blocks (ADR 0016).

2.5 Gates relative to the change, and a loop that may only tighten

The gate judges the branch, not the estate. It runs the same validators on the working tree and on the merge-base, over the same scope. A finding present in both is reported as pre-existing and never blocks. A line moved by an edit is not new. Without git the gate gets stricter, never looser, and counts every finding as new (ADR 0018).

Every fix teaches the next run. When the gate finds an error the branch introduced, it suggests `/dgf-fix`. That skill reproduces the error, fixes it inside the plan's scope, confirms the fix with the same check and records a patch: problem, root cause, solution and prevention. When asked, `/dgf-evolve` distils patches into project rules for ten skills. A rule may add checks. It may never relax a stop condition, an exit code, a gate's computed status or an ownership rule. Anyone who can commit could write one, so a script checks every rule before a skill reads it, and refuses one that grants a tool, installs something, or passes a script a flag that would skip a check (ADR 0024).

Blast radius is a report, not a gate. `/dgf-audit` builds a reference graph per application, `webasm` included, and names the processes that reach a changed file, so the team knows what to regression-test before a shared workflow changes. Every reach report states what a static walk cannot see (ADR 0027 §5, §8).

3. What changes for a delivery team

A developer on an inspections estate is asked to hide a fee panel until the payment gateway confirms payment. The fee data source lives in `webasm` and is shared by every application's payment page.

Plan. `/dgf-plan` writes the plan on its own branch. It lists `zims` and `webasm`, and a script rejects the plan until it says why `webasm` is touched (ADR 0017; ADR 0028 rule 3). One task is marked as code, because no configuration verb reads the gateway callback, and the plan must say so. Another branch's plan touching the same files is a warning (ADR 0009). Before the shared data source is edited, `/dgf-audit` lists the processes in each application that reach it.

Implement. `/dgf-implement` works task by task and ticks each one, so a session can stop and resume. It writes JSON only where the parity table says the runtime reads it, and keeps the five legacy types in XML (ADR 0010). It checks each task's files as it goes.

Verify, fix, commit. Suppose a transition was left pointing at a renamed state. The gate reports that one new error, lists the estate's older findings as pre-existing, and suggests `/dgf-fix`. On a copy of DGF's samples, one introduced dead transition came back as exactly one new error among 2,552 pre-existing findings, in about five seconds (ADR 0018). `/dgf-fix` repairs it and records a patch. `/dgf-qa` writes a manual test plan for every process the change reaches (ADR 0032), and `/dgf-commit` commits by workspace.

Review. The reviewer gets a branch whose plan is checked, whose scope is enforced and whose gate result was computed by a script. Review time goes on the question only a person can answer: is this the right change?

Question	Today	With dgf-factory
Does the runtime read this JSON at all?	The reviewer's memory	Decided per file from the parity table (ADR 0011)
Does every transition name a real state?	Nothing reports it before run time	Blocked at the gate (ADR 0014)
Is this finding ours, or was it already there?	Unknown, so all get ignored	New findings block; older ones are listed (ADR 0018)
Did a `webasm` change break another application?	Found when that application is tested	Reach and references resolved per application (ADR 0014, 0027)
Which processes need regression testing?	The tester's judgement	Listed, with the limits of a static walk stated (ADR 0027 §8)
What did we learn from the last fix?	Someone's memory	A patch, then a checked rule (ADR 0024)
Did the agent stay inside the task?	Read the whole diff	Changed files checked against the plan (ADR 0018)

For a new application, `/dgf-scaffold` turns a short design (modules, entities, sign-in method) into a solution, a database project, workspaces and a local container stack. It works from shipped templates and writes only into an empty folder. A script checks secrets, structure, routes and configuration before

anything is reported (ADR 0027 §7).

4. Why it is credible

The rules were tested against DGF's own estate before they were trusted. The first process rules failed DGF's 23 sample processes. Their successor blocked 62 of 172 workflow references, most of which the runtime loads. Only the third version, which resolves references by the engine's own rules, was kept (blueprint open question 5; ADR 0014).

The validators run over DGF's samples. A maintainer tool runs every validator over DGF's sample estate. Every remaining error must be excused by an entry citing the runtime code that explains it, and an excuse that no longer matches a finding fails the run. At the recorded baseline (27 September 2026) it covered 362 process and workflow files, 3,116 configuration files and 1,214 model files. The verdict was clean: 100 errors excused one by one, plus 9 processes the samples call but do not contain (ADR 0014; see Measurements).

Every blocking finding has a failing example. The known-bad corpus holds 99 synthetic cases, one for each blocking finding and each "could not run" path. Each must produce exactly its stated exit code and findings, and no other error.

Tests. 1,339 Python unit tests and 87 installer tests pass. One test fails when a skill quotes a finding code or a flag that no script has, so the prompts cannot drift from the scripts.

Decisions are recorded, costs included. There are 34 decision records, 21 in force. Each has dated evidence, the alternatives rejected and a mandatory list of negative consequences.

5. What it deliberately does not do

It does not	Why, and what happens instead
Write C#, TypeScript, Angular, SQL or plugin assemblies in the change pipeline	It composes configuration. A task needing other code stops and is reported; `/dgf-model` names the database work and writes no SQL (ADR 0010 §3, ADR 0027 §3). The one exception is the scaffold, which works from templates and only into an empty folder (ADR 0027 §7)
Run a process, or test in a browser	A process cannot run without a case record and the database. The claim is bounded: the process matches the grammar the runtime reads and is free of the defects listed. A process can pass every check and still fail in the browser; runtime confidence stays with DGF's Playwright suite and with people (ADR 0014 §5)
Check roles	Roles are database rows; nothing in a workspaces root declares one. Role names are listed, not checked (ADR 0027 §6; knowledge base: permissions §1)
Give a complete blast radius	A static walk cannot see run-time names, processes chosen from the database, or entry points outside processes. Every reach report says so (ADR 0027 §8; knowledge base: reference graph §3)
Gate on the DGF release yet	`/dgf` records the estate's version and warns when it is newer than the knowledge. No fact carries a version range yet, so the version decides nothing (ADR 0019)

It does not	Why, and what happens instead
Block on every workflow reference	Sub-workflow calls and workflow-to-form references are reported by the audit, not blocked. Unreachable states are advisory (ADR 0014, ADR 0027)
Migrate a non-DGF system, or upgrade DGF	No migration skill exists. An upgrade is a guided route: release notes, the new version declared, the whole-root gate (ADR 0034)
Run tasks in parallel outside Claude Code	Parallel, worktree-isolated execution is Claude Code only (ADR 0033). On other tools, interactive questions become plain instructions (ADR 0029 §8)
Promise that a scaffolded application runs as delivered	Its output builds and passes its own check, but has not run on a live DGF consumer stack: DGF publishes no consumer API or UI image, package feed or database baseline. Its workspace has run once, in a rehearsal, on DGF's showcase stack built from source (ADR 0027 §7.12; knowledge base: application layout §8)

6. Risks, costs and containment

- **The knowledge goes stale.** On 3 November 2026 the maintainer checks were run against a DGF 1.1.15 checkout; the knowledge is stamped 1.1.11. The drift check reported 147 changed sources among 565 tracked. The samples run was blocked by 52 errors no excuse covers, among them component types (`chart`, `autoComplete`) the shipped knowledge does not know. The checks did their job, but only when run (ADR 0013). **The knowledge must be refreshed before a pilot**, and for each DGF release an estate adopts.
- **The parity authority is a hand-maintained DGF wiki page.** Every parity claim cites one of its rows and its digest is tracked, but when the page is stale so is this tool (ADR 0011).
- **Common warnings get ignored.** The samples produce 43 app-dependent workflow warnings. Only new ones reach the gate, but a real failure can hide among them (ADR 0014).
- **An estate rule could relax a skill.** A script checks every rule before it is read, but the check is lexical: a relaxation in plain words passes it, so committed rules need review (ADR 0024).
- **The model may not follow its instructions.** Scripts decide what they can and compute the gate's status (ADR 0022). Planning, composing and judgement still rest on the model (blueprint, "The honest trade-off").
- **The gate has known blind spots.** A finding made worse without changing its message passes as pre-existing (ADR 0018). A wrongly declared DGF version is never checked against a deployment (ADR 0019).
- **Costs.** Node 18.3+ to install and Python 3.9+ for the validators, in a private environment per project (ADR 0029, ADR 0031). About 4.4 MB is committed to each project, and each update is a change to commit (ADR 0029). Every change needs a plan, even a one-line fix (ADR 0024). Code tasks get a weaker gate than configuration: the frontend tests, where they exist (ADR 0010). This team maintains validators DGF could arguably own (ADR 0014).

7. Decisions asked of the reader

1. **Pilot one brownfield estate.** One estate and one team for a fixed period, in Claude Code, where every feature is available. Measure findings caught at the gate against defects found later in test, patches recorded, and plan-to-merge time.

2. **Refresh the knowledge first.** Name a maintainer to re-vendor the knowledge base to the pilot estate's DGF release and settle the samples run to clean, and make that refresh part of adopting every DGF release.
3. **Own the estate's rules.** Six default rules ship (ADR 0028), and `/dgf-rules` writes each estate's rules file (ADR 0032). Name who approves new rules and reviews the project rules `/dgf-evolve` writes (ADR 0024).
4. **Distribution.** The tool is the internal npm package `@andrian-org/dgf-factory`, version 0.2.0, published to the organisation's private GitHub Packages registry (ADR 0029). Decide who may install it, and whether its releases follow DGF releases.
5. **The scaffold.** Allow `/dgf-scaffold` for real applications before DGF publishes the consumer images and feed it needs to run as delivered, or keep it to prototypes until then.
6. **Upstream.** Ask the DGF core team to own what this tool works around: parity as a queryable capability (ADR 0011), process cross-reference constraints (ADR 0014) and grammars that match the runtime (ADR 0016).

Sources

Every claim cites a dgf-factory decision record (ADR) or a section of its shipped DGF knowledge base. Knowledge base sections cited: README §1; schema families §1, §2, §6; component catalogue §1; JSON reader §1; composition specs §1.1, §4; reference graph §3; permissions §1; application layout §8. The blueprint is the design document whose open questions the ADRs answer.

ADR	Title
0009	One plan ledger at workspaces-root scope, partitioned by affected workspaces
0010	/dgf-implement composes configuration, modern JSON first, and writes JS or CSS only where configuration cannot
0011	Resolve the schema family first; format-coverage.md is the only runtime-parity authority
0013	Stamp every fact, gate only behavioural ones — provenance in a maintainer ledger
0014	Process verification — structure with a runtime-divergence allowance, semantics resolved the way the runtime resolves them
0016	Legacy XSDs other than process lag the runtime; their failures warn until each has a divergence list
0017	The plan file: a frontmatter header, per-task kind, reason and files, and edit-only global scripts
0018	A gate blocks on what the branch introduced: findings are compared with the merge-base
0019	The estate's DGF version is declared at setup and recorded in the pipeline config
0022	Every gate block is built by one library, its status computed from its entries, and it never claims a check it did not run — and the verify gate sends a finding the branch introduced to /dgf-fix
0024	The learning loop: /dgf-fix records patches inside a plan's scope, and /dgf-evolve writes overrides a script checks, which may only tighten a skill — read by ten skills
0026	Brownfield-first, scoped to the whole workspaces root — and the scaffold generates a new application from an empty folder
0027	The DGF-specific skills write only inside a plan's scope, check the model in the gate, and report blast radius from a reference graph resolved per application — and /dgf-scaffold creates an application from an empty folder
0028	Six shipped defaults for dotGov DGF systems, and sign-in chosen per estate
0029	Delivered as an npm CLI that installs into a project, with an AI Factory-derived architecture

ADR	Title
0031	Validators run on Python with lxml and jsonschema in a project virtual environment, and read JSON the way the runtime does
0032	The quality and authoring skills report rather than gate, each writes one artifact of its own, and the rules file is checked by a script before any skill reads it
0033	Parallel execution: a Claude Code coordinator dispatches one task per worktree worker, and scripts decide the layers, the sync and the merge
0034	The security scan ships in the runtime behind /dgf-security, and migration and upgrade are guided routes, not skills

Measurements

Figure	Source
66 components with a JSON schema; five counts (34, 67, 56, 59, 66)	Knowledge base: component catalogue §1
67 parity rows: 57 supported, 7 partial, 2 not supported, 1 struck	Knowledge base: schema families §6, counted
69 JSON schemas and 9 XSDs vendored	Knowledge base: schema families §1
42 sample DataTables with `showHeaders`; 0 of 230 sample files pass the schemas read literally	ADR 0031
`workflow.xsd` rejects 211 of 341 samples; grid-form 54%, table-view 43%	ADR 0016, measured 24 September 2026
23 sample processes; 62 of 172 references blocked by the earlier rules	Blueprint open question 5; ADR 0014
1 new error among 2,552 pre-existing findings, about 5 seconds	The pipeline documentation, measured on a copy of DGF's samples (ADR 0018)
362 / 3,116 / 1,214 files; 100 errors excused, 9 external processes; clean; 43 app-dependent warnings	The samples-run baseline of 27 September 2026, recorded with the tool's list of excuses
99 known-bad cases; 1,339 Python and 87 installer tests passing	Counted and run 3 November 2026
22 skills; 16 install targets; version 0.2.0, published 27 October 2026	The package and its publishing run
34 ADRs, 21 in force	The ADR index
147 of 565 sources drifted; 52 unexcused errors at DGF 1.1.15	Drift check and samples run against a DGF checkout of 2 November 2026, run 3 November 2026
Scaffolded workspace run on DGF's showcase stack	The demo rehearsal log, 4 October 2026
About 4.4 MB committed per project	ADR 0029